In [1]:
```
import pandas as pd

import altair as alt
alt.data_transformers.enable('vegafusion')

import warnings
warnings.filterwarnings('ignore', module='altair')
```

The dataset in the shown in the repository and shiny app was obtained from the Walmart-Sales-Data-Analysis public repository by Mohammed Shehbaz Damkar.

Peak at data structure types and head/tail values

In [2]:
```
walmart_df = pd.read_csv('../data/raw/walmart_sales_data.csv')
walmart_df
```

Out[2]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Ta
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.
...	...	...	...	...	...	...	...	...	
995	233-67-5758	C	Naypyitaw	Normal	Male	Health and beauty	40.35	1	2
996	303-96-2227	B	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.
997	727-02-1313	A	Yangon	Member	Male	Food and beverages	31.84	1	1.
998	347-56-2442	A	Yangon	Normal	Male	Home and lifestyle	65.82	1	3
999	849-09-3807	A	Yangon	Member	Female	Fashion accessories	88.34	7	30

1000 rows × 17 columns

Check for NA values counts, none exist

In [3]:
```
walmart_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Invoice ID               1000 non-null   str
 1   Branch                   1000 non-null   str
 2   City                     1000 non-null   str
 3   Customer type            1000 non-null   str
 4   Gender                   1000 non-null   str
 5   Product line             1000 non-null   str
 6   Unit price               1000 non-null   float64
 7   Quantity                 1000 non-null   int64
 8   Tax 5%                   1000 non-null   float64
 9   Total                    1000 non-null   float64
 10  Date                     1000 non-null   str
 11  Time                     1000 non-null   str
 12  Payment                  1000 non-null   str
 13  cogs                     1000 non-null   float64
 14  gross margin percentage  1000 non-null   float64
 15  gross income             1000 non-null   float64
 16  Rating                   1000 non-null   float64
dtypes: float64(7), int64(1), str(9)
memory usage: 205.6 KB
```

Glance at common stats for each column number of unique values, most common values, min/max etc.

In [4]:
```
walmart_df.describe(include='all')
```

Out[4]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Qu
count	1000	1000	1000	1000	1000	1000	1000.000000	1000.0
unique	1000	3	3	2	2	6	NaN	
top	750-67-8428	A	Yangon	Member	Female	Fashion accessories	NaN	
freq	1	340	340	501	501	178	NaN	
mean	NaN	NaN	NaN	NaN	NaN	NaN	55.672130	5.5
std	NaN	NaN	NaN	NaN	NaN	NaN	26.494628	2.9
min	NaN	NaN	NaN	NaN	NaN	NaN	10.080000	1.0
25%	NaN	NaN	NaN	NaN	NaN	NaN	32.875000	3.0
50%	NaN	NaN	NaN	NaN	NaN	NaN	55.230000	5.0
75%	NaN	NaN	NaN	NaN	NaN	NaN	77.935000	8.0
max	NaN	NaN	NaN	NaN	NaN	NaN	99.960000	10.0

Make the date column a pd.DateTime object

In [5]:
```
walmart_df['Date'] =  pd.to_datetime(walmart_df['Date'], format='%Y-%m-%d')
walmart_df['Date']
```

Out[5]:
```
0      2019-01-05
1      2019-03-08
2      2019-03-03
3      2019-01-27
4      2019-02-08
          ...
995    2019-01-29
996    2019-03-02
997    2019-02-09
998    2019-02-22
999    2019-02-18
Name: Date, Length: 1000, dtype: datetime64[us]
```

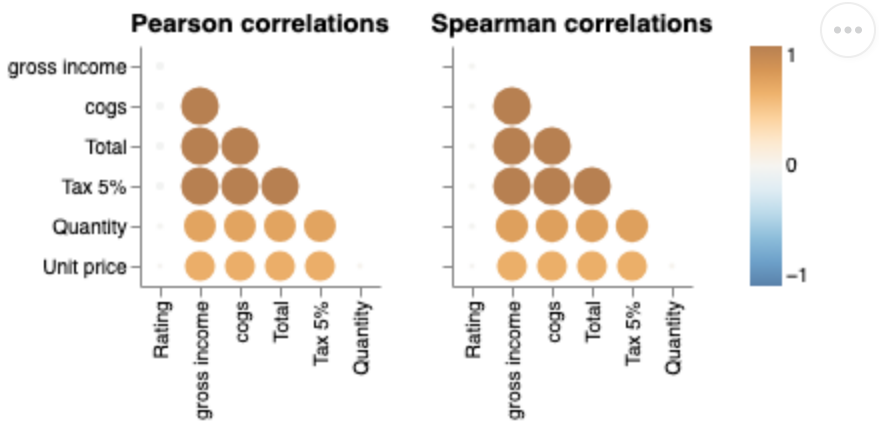
Check What Numerical Data is Correlated

In [6]:
```
import altair_ally as aly

aly.alt.data_transformers.enable('vegafusion')

aly.corr(walmart_df)
```

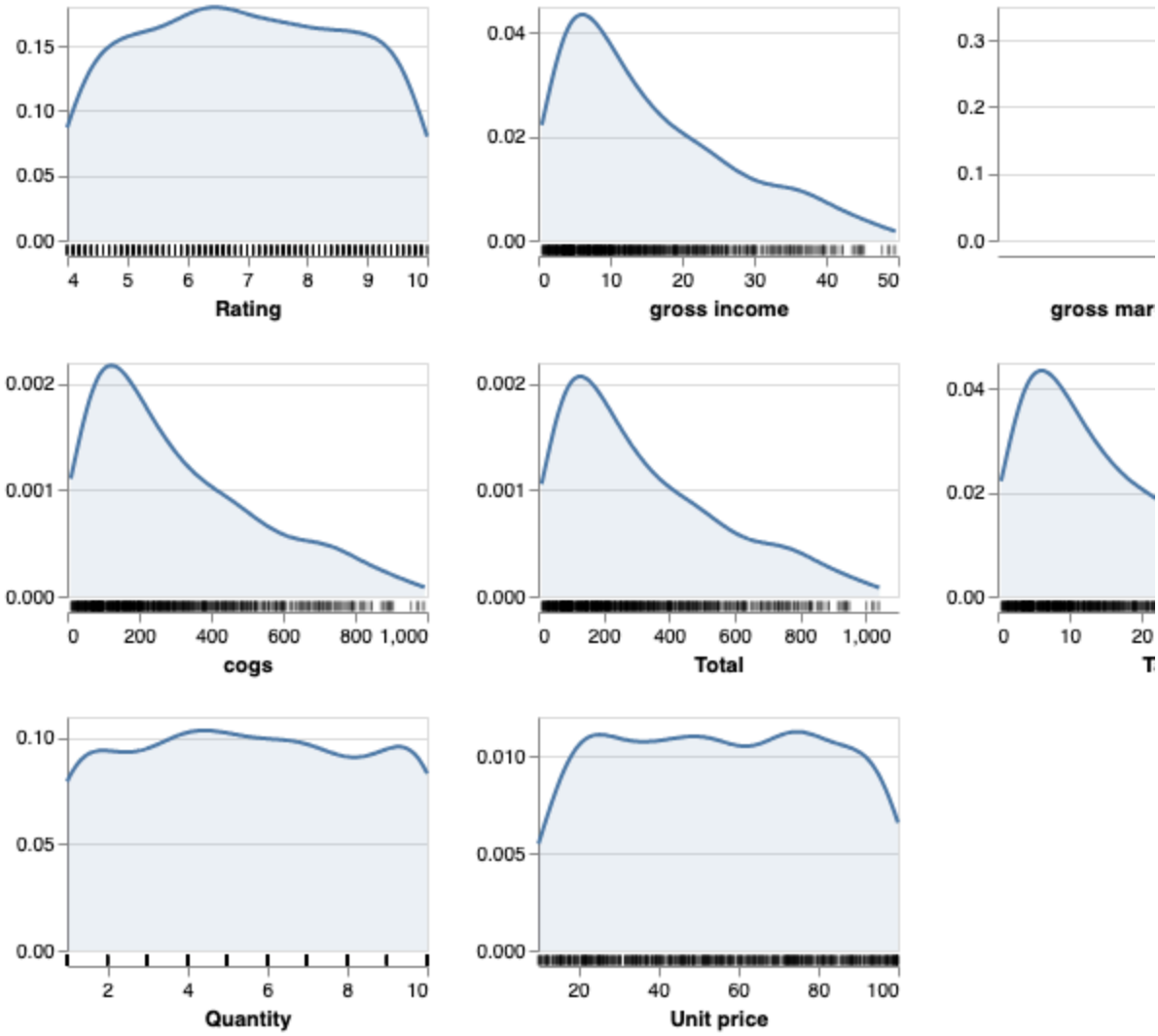
Out[6]:



Check Distributions of Numerical Data

In [7]:
```
aly.dist(walmart_df)
```

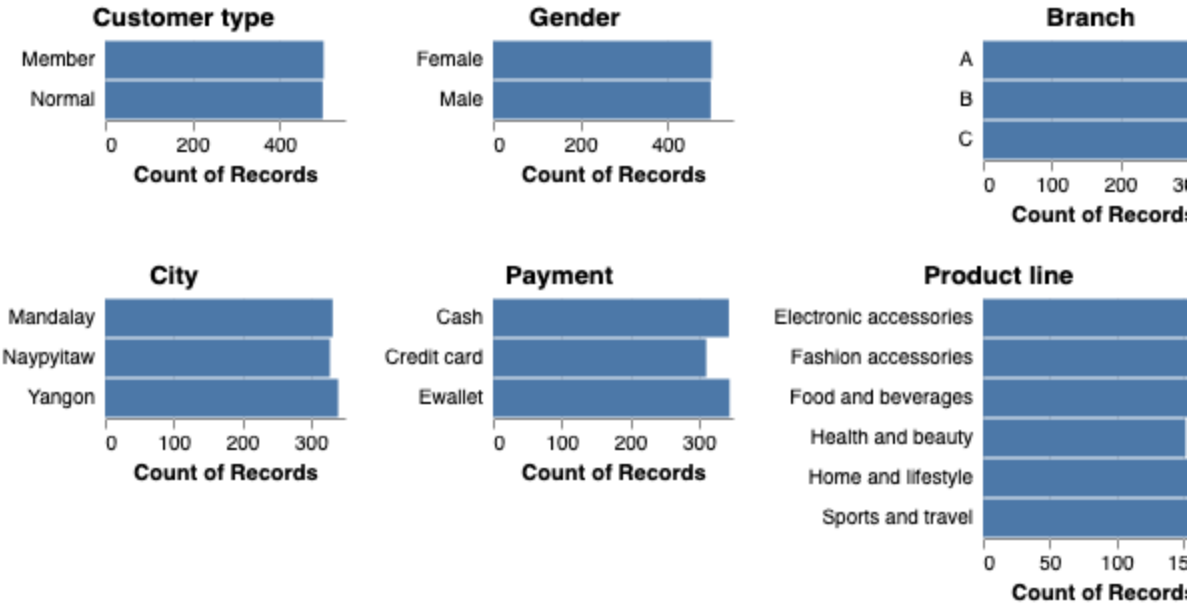
Out[7]:



Check counts of categorical data

In [8]:
```
aly.dist(walmart_df.drop(['Invoice ID', 'Time'], axis=1), dtype='object')
```

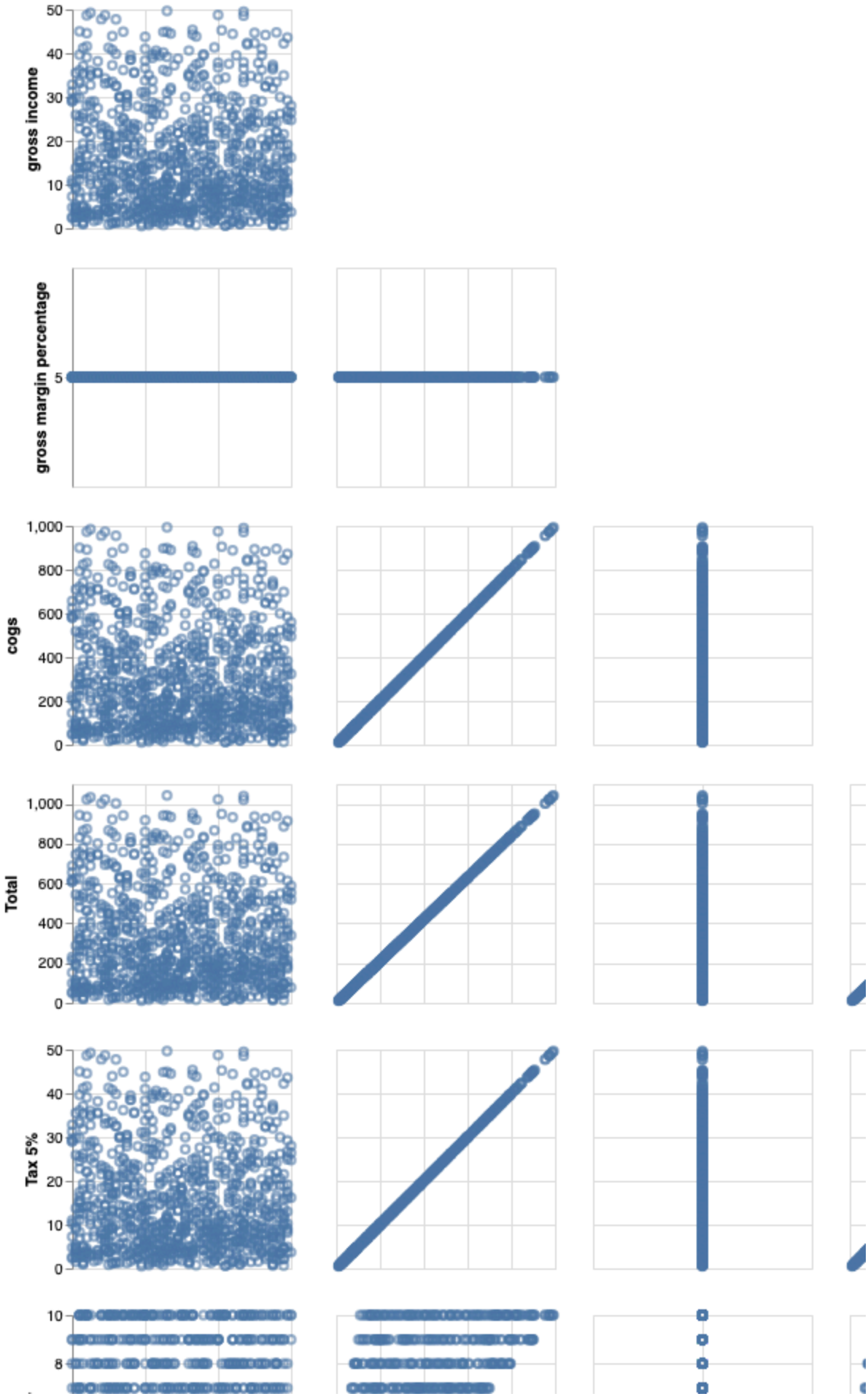
Out[8]:

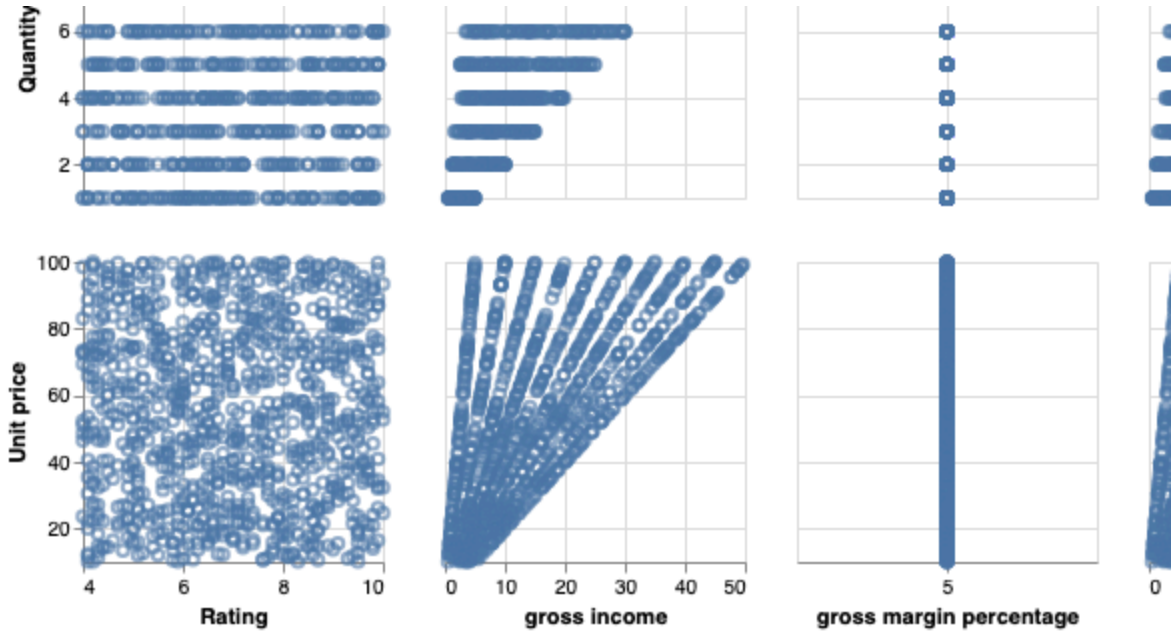


Check for any obvious relationships of interest for data insight on dashboard

In [9]:
```
aly.pair(walmart_df)
```

Out[9]:


gross income
gross margin percentage
cogs
Total
Tax 5%
50
40
30
20
10
0
5
1,000
800
600
400
200
0
10
8



Differences of Interest Across Categories

Categories to Check KPIs Across:

- ['Branch' or 'City', 'Customer type', 'Gender', 'Product line', 'Payment']

KPIs:

- ['Unit price', 'Quantity', 'Total', 'cogs', 'gross income', 'Rating']

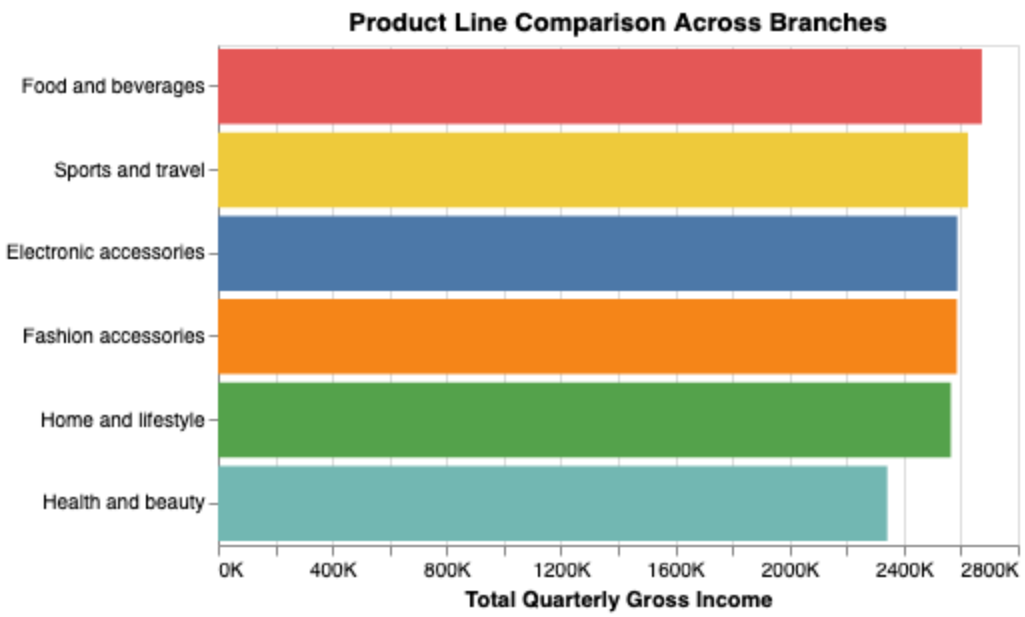
--- FINAL PLOTS BELOW ---

Note: After Exploring many different plot combinations with many variable combinations the following plots below we found to be most useful for dashboard and user scenarios

In [10]:
```
# Create the aggregate quarterly sales bar plot for each product line
bar = alt.Chart(walmart_df).mark_bar().encode(
    y = alt.Y('Product line:N', sort='-x', title=None),
    x = alt.Y('sum(gross income):Q',
              title='Total Quarterly Gross Income',
              axis=alt.Axis(labelExpr="datum.value + 'K'")),
    color= alt.Color('Product line:N', legend=None)
).properties(title = 'Product Line Comparison Across Branches',
             width = 400, height = 250)


# Save and display the bar plot
bar.save('../img/quarterly_product_sales.png', ppi=300)
bar
```

Out[10]:



In [11]:

```
# Make date the index for resampling by week
walmart_df = walmart_df.set_index('Date')

# Resample sales for each city by summing across each week to reduce plottin
# Note that if you resample on entire data frame gross income per city info
weekly1 = walmart_df[walmart_df['City']=='Yangon'
                     ].resample('W')['gross income'].sum()
weekly1 = weekly1.reset_index()
weekly1['City'] = 'Yangon'

weekly2 = walmart_df[walmart_df['City']=='Naypyitaw'
                     ].resample('W')['gross income'].sum()
weekly2 = weekly2.reset_index()
weekly2['City'] = 'Naypyitaw'

weekly3 = walmart_df[walmart_df['City']=='Mandalay'
                     ].resample('W')['gross income'].sum()
weekly3 = weekly3.reset_index()
weekly3['City'] = 'Mandalay'


# Concatenate the city data frames for sorting by color in line plot
weekly_df_city = pd.concat([weekly1.reset_index(),
                        weekly2.reset_index(),
                        weekly3.reset_index()])

# Create the weekly sales line plot for the cities
line = alt.Chart(weekly_df_city).mark_line().encode(
    x = alt.X('Date:T',
              axis=alt.Axis(labelAngle=45)),
    y = alt.Y('gross income:Q', title='Gross Income',
              axis=alt.Axis(labelExpr="datum.value + 'K'")),
              color='City:N'
).properties(width = 400, height = 250,
             title='Weekly Sales of Walmart Branches')
```

```
# Save and display the line plot
line.save('../img/weekly_city_sales.png', ppi=300)
line
```

Out[11]: